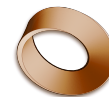


Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science



Year 11 2026

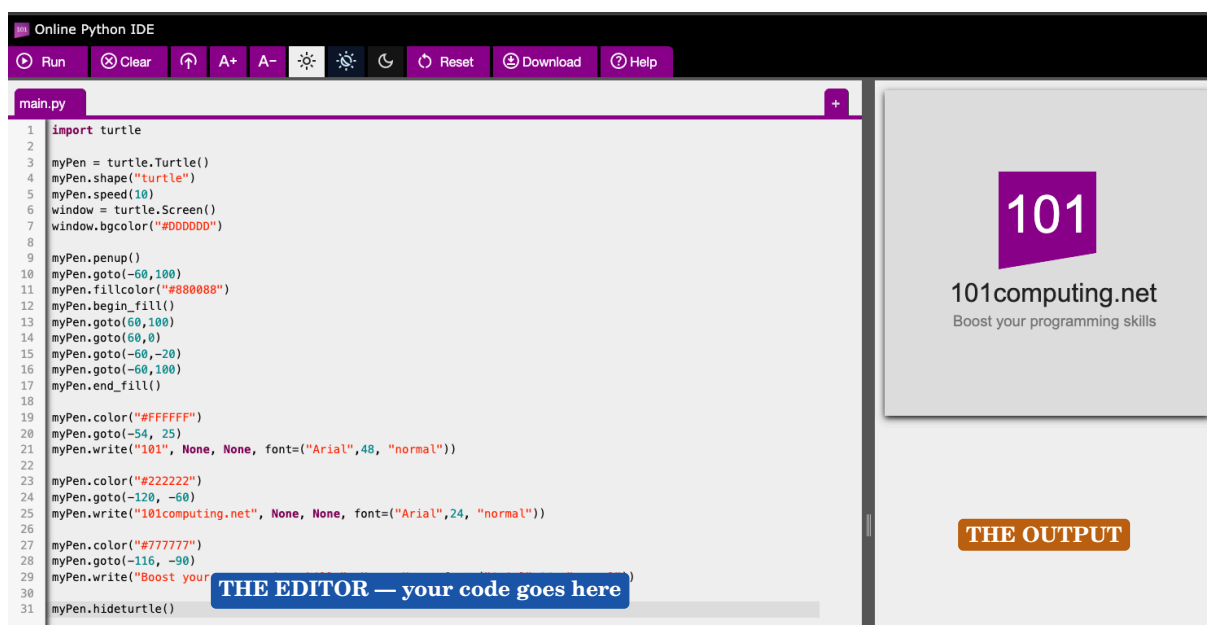
Programming 1 — IDE Setup

Your first Python — OCR J277 §2.2 and §2.5

Getting in — no account, no installing

1. Open any browser, on any device.
2. Go to 101computing.net/online-python-ide
3. Delete the demo code that is already there — see below.
4. Type your own code, then press .

No login, ever. Bookmark it — we work here all term.



Code on the left. Anything your program prints or draws appears on the right.

First job — empty it out

It does not open blank. **31 lines** of demo code are already there — a turtle program drawing the 101computing logo. That is their advert, not your program.

Delete all 31 lines: click in the code, press Ctrl+A (Cmd+A on a Mac), then Delete. You should be left with one empty line, numbered 1.

Watch out: Reset puts the demo straight back.

Learn the three parts by name

You are looking at an **IDE** — an Integrated Development Environment. It is not the language. Python is the language; the IDE is the workshop you write it in.

The editor The big area where you type. It colours your code to help you see it.

The run button  — starts your program. This is the **run-time environment**.

The output The panel on the right, where anything your program prints or draws appears — and where errors are reported. That error reporting is **error diagnostics**. (Many IDEs call this panel the *console*.)

Why bother with the names? Because §2.5.2 of your specification asks for them in the exam, and you are about to spend a year using all three. You may as well collect the marks while you learn to code.

Program 1 — First String

Type this exactly. Then press run.

```
print("Hello, world!")
```

print() puts something on the screen. The words inside the quote marks are a **string**.

VOCAB String (OCR §2.2.2) — a data type holding a sequence of characters. In short, text

Now try • Change the message to your own name.

Now try • Add a second print() line underneath. What order do they come out in?

Now try • Delete one of the quote marks and run it. *Read the error.* Then put it back.

Program 2 — First Variable

```
name = "Ada"
print("Hello, " + name)
```

name is a **variable** — a labelled box holding a value. You put something in with =, and get it back by using the label.

VOCAB Variable (OCR §2.2.1) — a named store in memory holding a value that *can* change while the program is running

VOCAB Assignment (OCR §2.2.1) — putting a value into a variable — what = does

Now try • Change what is in the box to your own name.

Now try • Add a second variable, age, and print it too.

Now try • What happens if you print name *before* the line that fills it?

Program 3 — First Input

```
name = input("What is your name? ")
print("Nice to meet you, " + name)
```

`input()` stops and waits for someone to type. Whatever they type goes into the variable.

VOCAB Integer (OCR §2.2.2) — a data type: a whole number, with no decimal part

The trap that catches everybody

`input()` **always** gives you text — even when the user types a number. So this looks right and is wrong:

```
age = input("Age? ")
print(age + 1)
```

Python refuses, because you asked it to add 1 to a piece of *text*. The fix is to convert it to an integer with `int()`:

```
age = int(input("Age? "))
print(age + 1)
```

This is called **casting**. Try the broken version first, on purpose, and read what the console says. You will meet this error many times this year.

VOCAB Casting (OCR §2.2.2) — temporarily changing a value from one data type to another. `int()` casts text into a whole number

Now try • Ask for two numbers and print their total.

Now try • Ask the user for their name *and* their age, then print one sentence using both.

Program 4 — First Selection

```
age = int(input("How old are you? "))

if age >= 16:
    print("You can be left at home alone.")
else:
    print("You cannot.")
```

The indent matters. Python uses those four spaces to know which lines belong to the `if`. Get it wrong and it will tell you.

VOCAB Selection (OCR §2.2.1) — choosing between different paths — `if` and `else`

Now try • Change the number 16 and re-run.

Now try • Remove the indent on one line. Read the error, then undo it.

Now try • Add an `elif` branch for a third possibility.

Program 5 — First Iteration

```
for i in range(5):
    print("Line number", i)
```

Now try • Make it count to 20.
Now try • Why does it start at 0 and not 1?
Now try • Make it print your name 100 times. (Three characters change.)

Break it on purpose

Good programmers are not people who never make mistakes. They are people who read the error and know what it means. So go and cause some, deliberately, and write down what the console said.

Do this to a working program	What happened?	What did the console say?
Delete a closing bracket		
Spell print as prnt		
Remove the colon after if		
Add 1 to an input()		

Now name them. Errors come in three kinds. Decide which kind each of your four breakages was — *two of them are the same kind*.

VOCAB Syntax error (OCR §2.3.3) — code that breaks Python’s rules. It will not run *at all*

VOCAB Runtime error (OCR §2.3.3) — it starts running, then hits something impossible and stops

VOCAB Logic error (OCR §2.3.3) — it runs perfectly and gives the *wrong answer*

If you get that far — the challenge

Write a program that:

1. asks the user for their name;
2. asks them how many times they want to be greeted;
3. greets them that many times.

Everything you need is on the page in front of you. You will have to combine `input()`, `int()`, and a `for` loop.

Before you go

Save before you close the tab. Use **Download**, or paste your code into a document in your Drive — the editor only remembers it in *this browser, on this device*.

Paper 2 is half your GCSE, and it asks you to write code. Today was the first day of that — and none of it needed installing.

Now write it out — by hand

Get it working in the IDE first, then **copy it out here by hand**. Not busywork: **Paper 2 is written on paper** — no colours, no Run button. Watch your indents.

main.py

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30